

Rapport – Wave Function Collapse parallèle

Auteurs : Amine Braham, Antoine Fournaise

Cours : Modèles de programmation parallèle – M1 CHPS

Date : Mai 2026

Sommaire

1. Description du projet
2. Stratégie d'implémentation
3. Difficultés rencontrées
4. Analyse de performance
5. Conclusion

1. Description du projet

1.1 Le problème

L'algorithme **Wave Function Collapse** (WFC) est une technique de génération procédurale qui synthétise une grille de sortie à partir d'un échantillon d'entrée, en imposant que chaque sous-grille $N \times N$ de la sortie soit une sous-grille $N \times N$ de l'échantillon. C'est un problème de satisfaction de contraintes (CSP) résolu par propagation locale.

Étapes principales :

1. **Extraction des tuiles** : énumération des sous-grilles $N \times N$ de l'échantillon et calcul de leur fréquence d'apparition.
2. **Calcul des compatibilités** : pour chaque paire de tuiles (t_1 , t_2) et chaque offset (ox , oy), déterminer si t_1 à la position p et t_2 à la position $p+(ox,oy)$ sont superposables sans conflit sur leur zone de chevauchement.
3. **Initialisation** : chaque cellule de la grille de sortie peut accueillir n'importe quelle tuile (état "superposé").
4. **Boucle de résolution** :
 - Calculer l'**entropie** de chaque cellule (nombre de tuiles encore possibles)
 - Sélectionner la cellule de plus faible entropie > 1 (heuristique MRV)
 - **Collapser** : choisir aléatoirement une tuile pondérée par sa fréquence
 - **Propager** les contraintes aux voisins via un BFS de type AC-3
5. Répéter jusqu'à ce que toutes les cellules soient collapsées (succès) ou qu'une contradiction apparaisse (échec).

1.2 Pourquoi paralléliser

Le calcul de compatibilité est en $O(K^2 \cdot (2N-1)^2 \cdot N^2)$ où K est le nombre de tuiles uniques – non négligeable pour de grandes grilles. La résolution elle-même appelle l'entropie sur toutes les cellules à chaque itération, et propage des contraintes sur des sous-zones potentiellement larges. Sur des grilles 256×256 ou 512×512 , le temps série dépasse la seconde.

2. Stratégie d'implémentation

2.1 Architecture générale

Le code est en C++17 templaté pour permettre :

- des **tuiles de taille N** quelconque (paramètre template `int N`)
- des **valeurs de cellule** de type quelconque (paramètre template `T`)

Trois fichiers d'en-tête principaux :

Fichier	Contenu
<code>include/grid.hpp</code>	Conteneur 2D générique avec <code>load</code> / <code>save</code> / <code>print</code>
<code>include/tile.hpp</code>	Tuile $N \times N$, extraction, test de compatibilité
<code>include/wfc.hpp</code>	Classe <code>WFC<N,T></code> + déclarations des deux solveurs

Les implémentations des solveurs sont dans `src/wfc_serial.hpp` et `src/wfc_omp.hpp` (séparés pour la lisibilité, inclus depuis `wfc.hpp`).

2.2 Représentation de l'état

L'**onde** (ensemble des configurations possibles) est représentée par :

```
using Wave = std::vector<std::vector<bool>>>;  
// wave[cell][t] = true ssi la tuile t est encore possible pour la cellule cell
```

Pour chaque cellule, un vecteur de booléens de taille `num_tiles` indique quelles tuiles restent compatibles avec les contraintes propagées. L'entropie est simplement le nombre de bits à `true`.

2.3 Table de compatibilité précomputée

Pour éviter de recalculer les compatibilités à chaque propagation :

```
// compatible[t1][oy+OFF][ox+OFF] = liste des t2 tels que t1 à p  
// et t2 à p+(ox,oy) sont compatibles  
std::vector<std::vector<std::vector<std::vector<int>>>>> compatible;
```

avec $OFF = N-1$ et $DIM = 2N-1$ (pour $N=2$, 9 offsets dont 8 utiles).

2.4 Version série

La résolution série suit la définition canonique :

```
tant que il reste des cellules non collapsées :  
  cell ← argmin(entropie) parmi les cellules d'entropie > 1  
  si entropie(cell) == 0 : échec  
  tile ← choix pondéré dans wave[cell]  
  collapse(cell, tile)  
  propager(cell) # BFS qui élague les wave[voisin]
```

La propagation utilise une file FIFO standard. Pour chaque cellule défilée, on examine ses voisins dans la fenêtre $(2N-1)^2$ et on retire de leur wave toutes les tuiles non supportées par au moins une tuile encore possible sur la cellule source.

2.5 Version parallèle OpenMP – stratégie finale

Trois axes de parallélisation possibles :

Phase	Parallélisable ?	Granularité
Recherche min-entropie	Oui (réduction)	$O(\text{rows} \cdot \text{cols})$ cellules indépendantes
Collapse	Non (RNG global)	–
Propagation BFS	Partiellement	$O(8)$ tâches par étape

Choix d'architecture : *une seule région parallèle* englobant toute la boucle principale, pour amortir le coût de création/destruction des threads (voir §3.3).

```

#pragma omp parallel
{
    while (!done) {
        #pragma omp for nowait
        for (cell : 0..total_cells-1)    // entropie
            ...
        #pragma omp critical { reduce }
        #pragma omp barrier

        #pragma omp single    // un seul thread fait collapse + BFS
        {
            collapse(...);
            while (!bfs.empty()) {
                src ← bfs.front();
                for (j : voisins de src)
                    #pragma omp task { propage src→dst_j }
                #pragma omp taskwait
                update bfs queue
            }
        }
    }
}

```

Invariant de sûreté (sans verrou) : pour une source `src` fixée, les voisins `dst_j = src + (oy_j, ox_j)` sont **tous distincts** car les offsets sont distincts. Les tâches écrivent donc dans des cellules disjointes de `wave[]` et `wave[src]` reste en lecture seule pendant toute la propagation issue de `src`. Aucun verrou n'est nécessaire sur `wave`.

3. Difficultés rencontrées

3.1 Première version – critical section globale

L'implémentation initiale enveloppait toute la mise à jour de `wave` dans un `#pragma omp critical(wave_update)`. Cela sérialisait toutes les tâches en un mutex unique, annulant complètement la parallélisation. La correction a nécessité une analyse fine pour prouver formellement que les écritures concurrentes étaient sûres sans verrou (cf. invariant de sûreté ci-dessus).

3.2 BFS visité vs en file

Première tentative de correction : ajouter un tableau `visited` pour éviter de retraiter la même cellule. **Bug** : dans WFC, une cellule peut nécessiter plusieurs passes de propagation si ses voisins sont modifiés après son premier traitement. La version série ne tracke pas le visité non plus, elle laisse simplement la file accepter des doublons. La bonne sémantique est un indicateur `in_queue` qui est *réinitialisé au défilement*, autorisant la remise en file si la cellule change ensuite.

3.3 Overhead de création des régions parallèles

Initialement, `#pragma omp parallel` était placé dans la boucle BFS, donc créé/détruit à chaque étape. Le **réveil de threads dormants** sur un nœud ARM coûte $\sim 0.1\text{--}1$ ms par région. Pour une grille 128×128 nécessitant des milliers d'étapes BFS, l'overhead atteint plusieurs secondes – **plus que le calcul utile lui-même**. La correction (région unique englobant toute la boucle, voir §2.5) a divisé par 2 le temps à $T=16$ mais n'a pas suffi à inverser l'anti-scalabilité globale (voir §4).

3.4 Bugs annexes

- **Makefile + SLURM** : le `make clean` supprimait `results/*.out`, effaçant le fichier de sortie SLURM en cours d'exécution. Solution : déplacer les logs SLURM dans un répertoire `logs/` distinct.
- **Architecture ARM** : les binaires compilés sur le nœud de login (x86) ne tournent pas sur les nœuds de calcul (ARM aarch64). La compilation doit se faire dans le job lui-même.

4. Analyse de performance

Toutes les mesures sur un nœud Romeo `armgpu` (architecture ARM aarch64, 32 cœurs alloués). Médiane sur 3 exécutions, graine 42, échantillon `binary_5x5.txt` (8 tuiles uniques).

4.1 Référence série

Grille	Temps série
16×16	0.002 s
32×32	0.004 s
64×64	0.063 s
128×128	0.140 s

Croissance proche du quadratique en nombre de cellules, conforme à la complexité théorique $O(R\cdot C\cdot K)$ où $K = \text{num_tiles}$.

4.2 Scalabilité en threads (binary 128×128)

Comparaison des deux architectures parallèles essayées :

Threads	V1 (région par étape BFS)	V2 (région unique)	Speedup V2 vs série
T=1	0.054 s	0.045 s	3.1×
T=2	0.985 s	1.218 s	0.12×
T=4	1.714 s	1.690 s	0.08×
T=8	3.767 s	2.852 s	0.05×
T=16	15.506 s	7.486 s	0.019×
T=32	27.068 s	27.956 s	0.005×

Observations :

1. **Anti-scalabilité confirmée** : ajouter des threads dégrade monotoniquement les performances dans les deux versions.
2. **V2 améliore significativement les hauts comptes de threads** : à T=16, V2 est 2.1× plus rapide que V1 grâce à l'élimination de l'overhead de création de région à chaque étape BFS. Mais l'amélioration n'est pas suffisante pour rendre la parallélisation rentable.
3. **T=1 est plus rapide que la version série pure** (0.045 s < série 128×128 pour binaire). L'écart n'est pas dû à la parallélisation (un seul thread) mais probablement à des effets de cache lors de la mesure.

4.3 Pourquoi l'anti-scalabilité ?

Plusieurs facteurs concourent :

(a) Granularité des tâches trop fine. Pour N=2, chaque étape BFS génère au plus 8 tâches. Chaque tâche effectue $\sim \text{num_tiles} \times |\text{compatible}| \approx 64$ comparaisons booléennes – soit quelques **nanosecondes** de calcul utile. L'overhead OpenMP par tâche (création, scheduling, taskwait) est de l'ordre du **microseconde**. Le ratio travail/overhead est défavorable d'un facteur 1000.

(b) Dépendance séquentielle entre itérations. La boucle principale de WFC est intrinsèquement séquentielle : chaque collapse dépend de l'onde résultant de tous les collapses précédents. Seul l'intérieur d'une itération peut être parallélisé, et cet intérieur (entropie + propagation locale) est petit.

(c) Faux partage sur le tableau `result[]`. Dans la propagation, chaque tâche écrit `result[j]` avec j unique. Mais `result` étant un `vector<int>` contigu, jusqu'à 16 entrées tiennent sur une même ligne de cache de 64 octets. Avec 8 threads écrivant simultanément sur des indices voisins, la ligne de cache rebondit entre les cœurs – effet **false sharing** classique.

(d) Trafic de cohérence de cache sur `wave`. Toutes les tâches lisent `wave[src]` simultanément. Bien que les lectures concurrentes soient correctes, sur ARM avec NUMA léger, la diffusion d'une ligne de cache à plusieurs cœurs ajoute de la latence par rapport à un accès local.

4.4 Scalabilité en taille de grille (T=16, binaire)

Grille	Temps T=16	Cellules	Temps/cellule (µs)
32×32	0.528 s	1 024	516
64×64	1.307 s	4 096	319
128×128	3.824 s	16 384	233
256×256	21.647 s	65 536	330
512×512	75.098 s	262 144	286

Le temps par cellule **diminue jusqu'à 128×128** (les phases parallèles – notamment l'entropie en `parallel for` sur toutes les cellules – amortissent mieux leur overhead sur de grandes plages d'itérations) puis se stabilise autour de 280–330 µs/cellule. Cette stabilisation indique que l'on a atteint le coût marginal réel de chaque cellule sur cette implémentation : ~300 µs par cellule, dominé par l'overhead OpenMP plutôt que par le calcul utile (qui prendrait ~1 µs en série).

4.5 Scalabilité en taille (T=16, échantillon multi-valeurs)

Avec l'échantillon `multi_8x8.txt` (3 valeurs distinctes) :

Grille	Temps T=16	Cellules	Temps/cellule (µs)
32×32	0.820 s	1 024	801
64×64	3.454 s	4 096	843
128×128	13.540 s	16 384	826
256×256	45.944 s	65 536	701

Le coût par cellule est **~3× plus élevé qu'en binaire** car :

- L'échantillon `multi_8x8` produit plus de tuiles uniques (donc des wave plus grandes, plus de comparaisons par cellule)
- La table `compatible[t][oy][ox]` est plus volumineuse → plus de pression cache lors des accès
- La propagation tend à durer plus longtemps avant convergence (plus de contraintes à satisfaire)

Le coût par cellule reste **stable autour de 700–840 µs** à travers les tailles, ce qui confirme que le surcoût est lié à la complexité par tuile et non à un problème d'échelle propre à OpenMP. La légère diminution observée à 256×256 (701 µs vs 826 µs en 128×128) traduit le même phénomène que pour les grilles binaires : sur de plus grandes tailles, les phases parallélisables (entropie) amortissent mieux leur overhead.

4.6 Que faudrait-il pour scaler ?

Plusieurs pistes nécessiteraient une refonte non triviale :

1. **Coarsening des tâches.** Regrouper plusieurs cellules sources dans une même tâche pour amortir l'overhead. Difficile car les sources successives du BFS ne sont pas indépendantes (elles peuvent modifier les mêmes destinations).
2. **Représentation de l'onde par bitset SIMD.** `std::vector<bool>` est un bitset mais sans opérations vectorielles efficaces. Un `bitset<K>` ou un `uint64_t` (si $K \leq 64$) permettrait des unions/intersections en une instruction, et accélérerait la propagation d'un facteur 10–100.
3. **Multiple restarts en parallèle.** Lancer T résolutions indépendantes avec des graines différentes et garder la première qui réussit – scalabilité parfaite mais hors cadre du sujet.
4. **GPU / Kokkos.** Déplacer entropie et compatibilité sur GPU. La propagation reste un goulot mais le calcul d'entropie sur 65k cellules pourrait s'exécuter en quelques μ s sur GPU.

5. Conclusion

Le projet a permis d'implémenter complètement l'algorithme WFC dans sa variante *overlapping*, en C++ templaté, avec une version série fonctionnelle et une version parallèle utilisant explicitement l'API task d'OpenMP comme demandé. La version multi-valeurs est obtenue gratuitement grâce à la généricité (paramètre template `T`).

Le résultat de performance le plus marquant – l'**anti-scalabilité** – est contre-intuitif mais explicable par l'analyse :

- WFC est **structurellement séquentiel** (dépendance entre collapses)
- La parallélisation ne peut s'appliquer qu'à l'intérieur d'une itération
- À l'intérieur, les unités de travail sont **trop petites** (~64 ops par tâche) pour amortir l'overhead OpenMP

Cette découverte est un résultat académiquement intéressant : tous les algorithmes ne sont pas adaptés à la parallélisation par tâches fines. Une parallélisation efficace de WFC nécessiterait soit une refonte algorithmique (multiple restarts, bitsets SIMD), soit un changement de modèle d'exécution (GPU).

Annexes

A. Configuration matérielle

- **Cluster** : Romeo (Université de Reims Champagne-Ardenne)
- **Nœud** : romeo-a057, contrainte `armgpu`
- **Architecture** : ARM aarch64, 32 cœurs

- **Mémoire** : 16 GiB
- **Compilateur** : GCC 11.4.1 avec `-O2 -fopenmp`

B. Reproduire les résultats

```
git clone https://github.com/Le-Amin/WFC_Algorithm.git
cd WFC_Algorithm
sbatch slurm/run_wfc.slurm    # ajuster --account
cat logs/wfc_*.out
```